МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № *3* по курсу «Операционные системы и архитектура компьютера»

Выполнила: Д. В. Новикова
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 21.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: освоить создание и ожидание потоков POSIX, реализовать параллельную обработку целочисленной матрицы и исследовать ускорение и эффективность алгоритма.

Задание: написать программу на языке C для обработки данных несколькими потоками. Максимальное число одновременно работающих рабочих потоков задаётся ключом запуска; его нужно показать средствами ОС и исследовать влияние размера данных и числа потоков на время работы.

Вариант 11: K раз наложить медианный фильтр на матрицу целых чисел. Размер окна задаёт пользователь.

Метод решения

Программа читает размеры и элементы матрицы из input.txt, затем получает нечётный размер окна и число применений K. Для каждого элемента она собирает значения квадратного окна, сортирует их и выбирает средний элемент отсортированного набора. На границах индекс за пределами матрицы заменяется ближайшим допустимым индексом, то есть крайние значения повторяются.

На каждом проходе строки результата делятся между p рабочими потоками на непересекающиеся диапазоны. Все потоки только читают исходную матрицу, а каждый поток пишет в собственные строки выходной матрицы и использует собственный временный массив окна. Главный поток ожидает все потоки через pthread_join, записывает готовую матрицу в output.txt и меняет входной и выходной буферы местами. После этого начинается следующий проход. Фактическое p равно минимуму из числа строк и предела, переданного через -t.

Для сравнения программа отдельно выполняет те же K проходов на копии исходной матрицы с одним рабочим потоком. Время измеряется по CLOCK_MONOTONIC; в измеряемый участок входят создание, вычисление и ожидание потоков, а запись матриц в файл не входит. Ускорение вычисляется как $S = T_1 / T_p$, эффективность — как $E = S / p$.

Описание программы

Реализация разделена на модули ввода, фильтра, измерения времени и управления запуском. Для потоков используется POSIX Threads; данные матрицы представлены int64_t, а размеры и индексы — size_t.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`main.c`	Параметры запуска, K проходов, запись результата и статистики.
`filter.c / filter.h`	Медианный фильтр, структура Worker, создание и ожидание потоков.
`matrix_io.c / matrix_io.h`	Чтение размеров и элементов матрицы из input.txt.

Файл	Назначение
benchmark.c / benchmark.h	Измерение времени и запуск с одним рабочим потоком.
CMakeLists.txt	Сборка программы и подключение POSIX Threads.
input.txt	Исходная матрица: размеры, затем элементы по строкам.
output.txt	Матрицы после проходов, времена, ускорение и эффективность.

Основные типы данных и функции

Worker содержит указатели на входную и выходную матрицы, размеры, величину окна, диапазон строк [first_row, last_row) и системный TID. filter_pass принимает массив Worker и число рабочих, возвращает 0 при успехе и -1 при ошибке. worker_main обрабатывает назначенные строки; при ошибке выделения памяти завершает поток через pthread_exit(work), а при успехе — через pthread_exit(NULL). Главный поток получает это значение через pthread_join. Функция clamped_index ограничивает индексы на границе матрицы; compare_int64 задаёт сравнение для qsort.

read_matrix выделяет память, считывает размеры и значения, возвращает 0 или -1. run_timed_pass добавляет длительность одного прохода к накопленному времени; measure_one_worker измеряет K проходов с одним потоком на копии матрицы. main проверяет ключ -t, параметры окна и K, записывает результат и освобождает память.

Используемые системные вызовы

В исходном коде вызываются функции POSIX и стандартной библиотеки; часть из них внутри libc обращается к ядру. Для показанного запуска strace зафиксировал следующие значимые вызовы.

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
`clone3`	Создание ядром потока при pthread_create; флаги включают CLONE_VM и CLONE_THREAD.
`futex`	Ожидание завершения рабочего потока при pthread_join и пробуждение ожидающего.
`gettid`	Получение системного TID для вывода сведений о рабочем потоке.
openat, read	Открытие input.txt и чтение исходных данных через функции stdio.
openat, write, close	Создание output.txt, запись результатов и закрытие файла.
mmap, munmap	Выделение и освобождение областей памяти библиотекой и средой выполнения.

clock_gettime(CLOCK_MONOTONIC) используется для измерений. В конкретной трассировке отдельной строки clock_gettime нет; это согласуется с получением времени через vDSO без перехода в ядро.

Ключевые фрагменты кода

После сбора и сортировки элементов окна рабочий поток записывает медиану только в назначенную ему ячейку.

Листинг 1 – Вычисление медианы (filter.c)

```
samples[used++] = work->input[source_row * work->cols + source_col];
qsort(samples, sample_count, sizeof(*samples), compare_int64);
work->output[row * work->cols + col] = samples[sample_count / 2];
```

Главный поток создаёт рабочих, затем ожидает каждого и проверяет переданное потоком значение ошибки.

Листинг 2 – Фрагменты создания и ожидания потоков (filter.c)

```
int error = pthread_create(&threads[created], NULL, worker_main,
                           &workers[created]);
if (error != 0) {
    fprintf(stderr, "pthread_create: %s\n", strerror(error));
    failed = 1;
    break;
}
void *thread_result = NULL;
int error = pthread_join(threads[i], &thread_result);
if (error != 0) {
    fprintf(stderr, "pthread_join: %s\n", strerror(error));
    failed = 1;
} else if (thread_result != NULL) {
    fprintf(stderr, "Cannot allocate median window for worker %zu\n", i + 1);
    failed = 1;
}
```

Результаты

Проверка корректности выполнена на матрице 3 × 3 с центральным выбросом 100, окном 3 × 3, K = 2 и пределом -t 2. Медиана каждой окрестности равна 1, поэтому после обоих проходов матрица состоит из единиц.

Листинг 3 – Проверочный запуск и результат

```
$ cd report/trace
$ printf '3\n2\n' | strace -f -o strace.txt ../../src/build/lab3 -t 2
Worker 1: PID=43139 TID=43144
Worker 2: PID=43139 TID=43145
Pass 1:               Pass 2:
1 1 1                 1 1 1
1 1 1                 1 1 1
1 1 1                 1 1 1
```

Для измерений использованы матрицы 400 × 400 и 800 × 800, окно 11 × 11, K = 3. Время T_p приведено для конкретного числа рабочих потоков. Каждая строка получена отдельным запуском; T_1 внутри него измеряется на копии входной матрицы. Числа округлены до трёх знаков после запятой.

Таблица 3 – Результаты для матрицы 400 × 400

Матрица / потоки	Время, мс	Ускорение	Эффективность
400 × 400 / 1	1354,263	1,000	1,000
400 × 400 / 2	692,239	1,987	0,994
400 × 400 / 4	373,588	3,634	0,908
400 × 400 / 8	390,321	3,409	0,426

Таблица 4 – Результаты для матрицы 800 × 800

Матрица / потоки	Время, мс	Ускорение	Эффективность
800 × 800 / 1	5250,713	1,000	1,000
800 × 800 / 2	2838,277	1,915	0,958
800 × 800 / 4	1461,366	3,574	0,893
800 × 800 / 8	1545,651	3,604	0,451

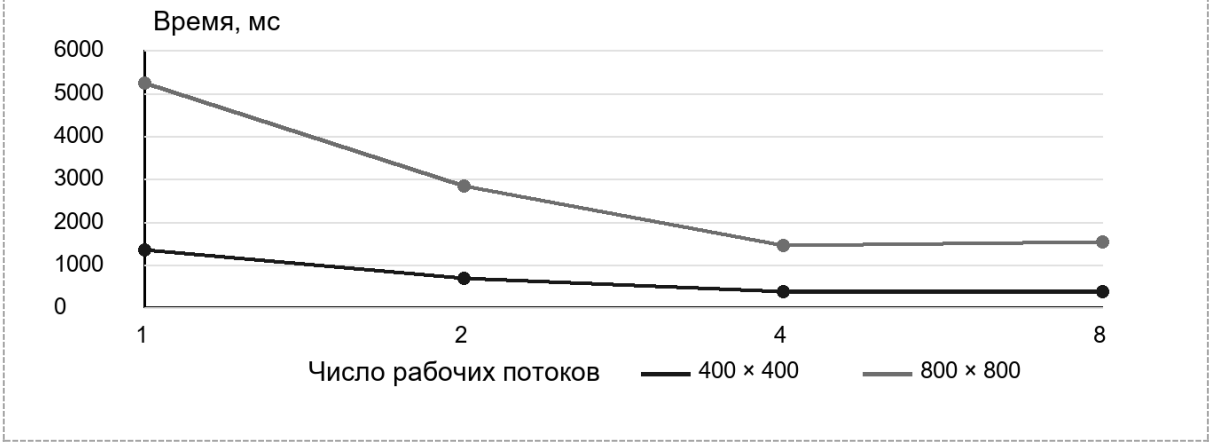


Рисунок 1 – Зависимость времени от размера матрицы и числа потоков

Переход от одного рабочего потока к четырём сокращает время для обеих матриц примерно в 3,6 раза. Восьмой поток не дал дальнейшего сокращения времени: создание дополнительных потоков, планирование и конкуренция за вычислительные ресурсы увеличивают накладные расходы. Ускорение зависит от размера данных: на матрице 800 × 800 абсолютный выигрыш заметно больше. Результат одного запуска может колебаться из-за фоновой нагрузки системы.

Для наблюдения за количеством потоков одновременно запускают watch -n 0.2 'ps -L -C lab3 -o pid,tid,stat,comm'. Во время вычисления с -t 4 видны пять строк с одинаковым PID и разными TID: главный поток и четыре рабочих. При переходе между проходами кратковременно может быть меньше строк.

Демонстрация работы strace

Проверочный запуск выполнен под strace с ключом -f, который включает созданные потоки. Ниже приведены строки фактической трассировки; полный вывод находится в приложении Б. Измерения таблиц 3 и 4 выполнены без strace.

Листинг 4 – Создание и ожидание потоков в strace

```
43139 openat(AT_FDCWD, "input.txt", O_RDONLY) = 3
43139 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|
CLONE_SIGHAND|CLONE_THREAD|...}, 88) = 43142
43139 futex(0x7a7deb7ff990, FUTEX_WAIT_BITSET|
FUTEX_CLOCK_REALTIME, 43142, NULL, FUTEX_BITSET_MATCH_ANY
 <unfinished ...>
43142 gettid() = 43142
```

openat открывает исходный файл. clone3 создаёт рабочий поток в общем адресном пространстве процесса; в полном выводе видно, что перед каждым из двух проходов создаются новые потоки. futex показывает ожидание завершения потока внутри pthread_join; gettid возвращает его системный идентификатор. Сокращения «...» в листинге 4 сделаны только для читаемости, полные строки сохранены в приложении Б.

Выводы

Реализован медианный фильтр для целочисленной матрицы с K повторениями и задаваемым размером окна. Независимые диапазоны строк позволили выполнять один проход несколькими потоками без гонки при записи, а pthread_join обеспечивает завершение прохода до смены буферов. Проверочный пример подтвердил результат фильтрации; ps и strace показали реальные потоки и вызовы ядра. На использованных данных наименьшее время получено при четырёх рабочих потоках; дальнейшее увеличение их числа не улучшило результат.

Приложение А. Исходный код программы

Ниже приведены исходные и заголовочные файлы программы, а также конфигурация сборки.

Листинг А.1 – Файл main.c

```
#include "filter.h"
#include "benchmark.h"
#include "matrix_io.h"

#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    size_t limit;

    if (argc != 3 || strcmp(argv[1], "-t") != 0 || argv[2][0] == '-' || sscanf(argv[2],
"%zu", &limit) != 1 || limit == 0) {
        fprintf(stderr, "Invalid input\n");
        return EXIT_FAILURE;
    }

    size_t rows = 0;
    size_t cols = 0;

    int64_t *input = NULL;
    if (read_matrix(&rows, &cols, &input) != 0)
        return EXIT_FAILURE;

    size_t window, iterations;

    printf("Enter window size:\n");
    if (scanf("%zu", &window) != 1 || window == 0 || window > rows || window > cols ||
        window % 2 == 0 || window > SIZE_MAX / window || window * window > SIZE_MAX /
sizeof(int64_t)) {
        fprintf(stderr, "Window must be odd and not larger than the matrix\n");
        free(input);
        return EXIT_FAILURE;
    }

    printf("Enter number of filter applications:\n");
    if (scanf("%zu", &iterations) != 1 || iterations <= 0) {
        fprintf(stderr, "K must be a positive number\n");
        free(input);
```

```
        return EXIT_FAILURE;
    }

    size_t count = rows * cols;
    // because we can have more workers than rows, we limit the number of workers to the
number of rows
    size_t worker_count = limit < rows ? limit : rows;

    int64_t *output = malloc(count * sizeof(*output));
    Worker *workers = malloc(worker_count * sizeof(*workers));

    if (output == NULL || workers == NULL) {
        perror("malloc");
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }

    double one_worker_seconds = 0.0;
    if (worker_count > 1 && measure_one_worker(input, output, rows, cols, window,
iterations, &one_worker_seconds) != 0) {
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }

    FILE *output_file = fopen("output.txt", "w");
    if (output_file == NULL) {
        perror("output.txt");
        free(input); free(output); free(workers);
        return EXIT_FAILURE;
    }

    fprintf(output_file, "\nMatrix %zu x %zu, window = %zu, K = %zu, workers = %zu\n",
rows, cols, window, iterations, worker_count);

    double total_seconds = 0.0;
    int failed = 0;

    for (size_t pass = 1; pass <= iterations; ++pass) {
        for (size_t i = 0; i < worker_count; ++i) {
            workers[i] = (Worker){
                .input = input, .output = output, .rows = rows, .cols = cols, .window =
window,
                .first_row = i * rows / worker_count, .last_row = (i + 1) * rows /
worker_count
            };
        }

        if (run_timed_pass(workers, worker_count, &total_seconds) != 0) {
            failed = 1;
            break;
        }

        fprintf(output_file, "Pass %zu:\n", pass);
        for (size_t row = 0; row < rows; ++row) {
            for (size_t col = 0; col < cols; ++col)
                fprintf(output_file, "%" PRId64 "%c", output[row * cols + col], col + 1
== cols ? '\n' : ' ');
        }

        if (ferror(output_file)) {
```

```
            perror("writing output.txt");
            failed = 1;
            break;
        }

        int64_t *temporary = input;
        input = output;
        output = temporary;
    }

    if (!failed) {
        if (worker_count == 1)
            one_worker_seconds = total_seconds;
        if (total_seconds <= 0.0) {
            fprintf(stderr, "Measured time is too short\n");
            failed = 1;
        } else {
            double speedup = one_worker_seconds / total_seconds;
            double efficiency = speedup / (double)worker_count;
            fprintf(output_file, "T1: %.9f s; T%zu: %.9f s\nSpeedup: %.6f; Efficiency:
%.6f\n", one_worker_seconds, worker_count, total_seconds, speedup, efficiency);

            for (size_t i = 0; i < worker_count; ++i)
                printf("Worker %zu: PID=%ld TID=%ld\n", i + 1, (long)getpid(),
(long)workers[i].tid);
        }
        if (ferror(output_file)) {
            perror("writing output.txt");
            failed = 1;
        }
    }
    if (fclose(output_file) != 0) {
        perror("closing output.txt");
        failed = 1;
    }

    free(input);
    free(output);
    free(workers);
    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Листинг А.2 – Файл filter.c

```
#define _GNU_SOURCE
#include "filter.h"

#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static int compare_int64(const void *left, const void *right)
{
    int64_t a = *(const int64_t *)left;
    int64_t b = *(const int64_t *)right;
    return (a > b) - (a < b);
}
```

```
// matrix edge processing
static size_t clamped_index(size_t center, size_t offset, size_t radius, size_t limit)
{
    if (offset < radius) {
        size_t distance = radius - offset;
        return center < distance ? 0 : center - distance;
    }
    size_t distance = offset - radius;
    return distance >= limit - 1 - center ? limit - 1 : center + distance;
}

static void *worker_main(void *argument)
{
    Worker *work = argument;
    size_t radius = work->window / 2;
    size_t sample_count = work->window * work->window;
    int64_t *samples = malloc(sample_count * sizeof(*samples));
    work->tid = gettid();

    if (samples == NULL)
        pthread_exit(work);

    for (size_t row = work->first_row; row < work->last_row; ++row) {
        for (size_t col = 0; col < work->cols; ++col) {
            size_t used = 0;
            for (size_t kr = 0; kr < work->window; ++kr) {
                size_t source_row = clamped_index(row, kr, radius, work->rows);
                for (size_t kc = 0; kc < work->window; ++kc) {
                    size_t source_col = clamped_index(col, kc, radius, work->cols);
                    samples[used++] = work->input[source_row * work->cols + source_col];
                }
            }
            qsort(samples, sample_count, sizeof(*samples), compare_int64);
            work->output[row * work->cols + col] = samples[sample_count / 2];
        }
    }
    free(samples);
    pthread_exit(NULL);
}

int filter_pass(Worker *workers, size_t count)
{
    pthread_t *threads = malloc(count * sizeof(*threads));
    if (threads == NULL) {
        perror("malloc threads");
        return -1;
    }

    size_t created = 0;
    int failed = 0;
    for (; created < count; ++created) {
        int error = pthread_create(&threads[created], NULL, worker_main,
&workers[created]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            failed = 1;
            break;
        }
    }
    for (size_t i = 0; i < created; ++i) {
```

```
        void *thread_result = NULL;
        int error = pthread_join(threads[i], &thread_result);
        if (error != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(error));
            failed = 1;
        } else if (thread_result != NULL) {
            fprintf(stderr, "Cannot allocate median window for worker %zu\n", i + 1);
            failed = 1;
        }
    }
    free(threads);
    return failed ? -1 : 0;
}
```

Листинг А.3 – Файл filter.h

```
#ifndef FILTER_H
#define FILTER_H

#include <stddef.h>
#include <stdint.h>
#include <sys/types.h>

typedef struct {
    const int64_t *input;
    int64_t *output;
    size_t rows;
    size_t cols;
    size_t window;
    size_t first_row;
    size_t last_row;
    pid_t tid;
} Worker;

int filter_pass(Worker *workers, size_t count);

#endif
```

Листинг А.4 – Файл matrix_io.c

```
#include "matrix_io.h"

#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int read_matrix(size_t *rows, size_t *cols, int64_t **matrix)
{
    FILE *file = fopen("input.txt", "r");
    if (file == NULL) {
        perror("input.txt");
        return -1;
    }

    int failed = 0;
    if (fscanf(file, "%zu %zu", rows, cols) != 2 || *rows == 0 || *cols == 0) {
        fprintf(stderr, "Invalid matrix size in input.txt\n");
        failed = 1;
    }
    if (!failed && (*rows > SIZE_MAX / *cols || *rows * *cols > SIZE_MAX /
sizeof(**matrix))) {
        fprintf(stderr, "Matrix is too large\n");
```

```
        failed = 1;
    }
    if (!failed) {
        *matrix = malloc(*rows * *cols * sizeof(**matrix));
        if (*matrix == NULL) {
            perror("malloc matrix");
            failed = 1;
        }
    }
    for (size_t i = 0; !failed && i < *rows * *cols; ++i) {
        if (fscanf(file, "%" SCNd64, &(*matrix)[i]) != 1) {
            fprintf(stderr, "Invalid matrix value at position %zu\n", i);
            failed = 1;
        }
    }

    if (fclose(file) != 0) {
        perror("Closing input.txt");
        failed = 1;
    }
    if (failed) {
        free(*matrix);
        *matrix = NULL;
    }
    return failed ? -1 : 0;
}
```

Листинг А.5 – Файл matrix_io.h

```
#ifndef MATRIX_IO_H
#define MATRIX_IO_H

#include <stddef.h>
#include <stdint.h>

int read_matrix(size_t *rows, size_t *cols, int64_t **matrix);

#endif
```

Листинг А.6 – Файл benchmark.c

```
#include "benchmark.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static double seconds_between(struct timespec start, struct timespec end)
{
    return (double)(end.tv_sec - start.tv_sec) +
           (double)(end.tv_nsec - start.tv_nsec) / 1000000000.0;
}

// mesures one pass of the filter with the given workers and adds the time to
total_seconds
int run_timed_pass(Worker *workers, size_t worker_count, double *total_seconds)
{
    struct timespec start, end;
    if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
        perror("clock_gettime start");
        return -1;
    }
```

```
    if (filter_pass(workers, worker_count) != 0)
        return -1;

    if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
        perror("clock_gettime end");
        return -1;
    }
    *total_seconds += seconds_between(start, end);
    return 0;
}

// measures one worker filtering the entire input and returns the total time in seconds
(for report)
int measure_one_worker(const int64_t *input, int64_t *output, size_t rows,
                       size_t cols, size_t window, size_t iterations,
                       double *seconds)
{
    size_t count = rows * cols;
    int64_t *copy = malloc(count * sizeof(*copy));
    if (copy == NULL) {
        perror("malloc copy");
        return -1;
    }
    memcpy(copy, input, count * sizeof(*copy));

    // read from copy and write to output, then swap them for the next iteration
    int64_t *source = copy;
    int64_t *target = output;
    *seconds = 0.0;
    int failed = 0;

    for (size_t pass = 0; pass < iterations; ++pass) {
        Worker worker = {
            .input = source, .output = target, .rows = rows, .cols = cols,
            .window = window, .first_row = 0, .last_row = rows
        };

        if (run_timed_pass(&worker, 1, seconds) != 0) {
            failed = 1;
            break;
        }

        int64_t *tmp = source;
        source = target;
        target = tmp;
    }
    free(copy);
    return failed ? -1 : 0;
}
```

Листинг А.7 – Файл benchmark.h

```
#ifndef BENCHMARK_H
#define BENCHMARK_H

#include "filter.h"

int run_timed_pass(Worker *workers, size_t worker_count, double *total_seconds);
```

```
int measure_one_worker(const int64_t *input, int64_t *output, size_t rows, size_t cols,
size_t window, size_t iterations, double *seconds);

#endif
```

Листинг А.8 – Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)

project(lab3 LANGUAGES C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

find_package(Threads REQUIRED)

add_executable(lab3
    main.c
    filter.c
    matrix_io.c
    benchmark.c
)

target_compile_definitions(lab3 PRIVATE _POSIX_C_SOURCE=200809L)
target_compile_options(lab3 PRIVATE -Wall -Wextra -Wpedantic)
target_link_libraries(lab3 PRIVATE Threads::Threads m)
```

Приложение Б. Полный вывод strace

Команда: printf '3\n2\n' | strace -f -o strace.txt ../../src/build/lab3 -t 2. Ниже приведён полный вывод файла strace.txt для этого запуска.

Листинг Б.1 – Полный вывод strace -f

```
43139 execve("../../src/build/lab3", ["../../src/build/lab3", "-t", "2"], 0x7ffc6f4e3948
/* 25 vars */) = 0
43139 brk(NULL)                         = 0x5d72c7169000
43139 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a7debb6f000
43139 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
43139 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
43139 fstat(3, {st_mode=S_IFREG|0644, st_size=42667, ...}) = 0
43139 mmap(NULL, 42667, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7a7debb64000
43139 close(3)                          = 0
43139 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
43139 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...,
832) = 832
43139 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
43139 fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
43139 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
43139 mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a7deb800000
43139 mmap(0x7a7deb828000, 1609728, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7a7deb828000
43139 mmap(0x7a7deb9b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b1000) = 0x7a7deb9b1000
43139 mmap(0x7a7deba00000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x7a7deba00000
43139 mmap(0x7a7deba06000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7a7deba06000
```

```
43139 close(3)                          = 0
43139 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a7debb61000
43139 arch_prctl(ARCH_SET_FS, 0x7a7debb61740) = 0
43139 set_tid_address(0x7a7debb61a10)   = 43139
43139 set_robust_list(0x7a7debb61a20, 24) = 0
43139 rseq(0x7a7debb62060, 0x20, 0, 0x53053053) = 0
43139 mprotect(0x7a7deba00000, 16384, PROT_READ) = 0
43139 mprotect(0x5d72a762a000, 4096, PROT_READ) = 0
43139 mprotect(0x7a7debba7000, 8192, PROT_READ) = 0
43139 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
43139 munmap(0x7a7debb64000, 42667)     = 0
43139 getrandom("\x86\x32\xb3\x8a\xf2\x93\xc7\xe5", 8, GRND_NONBLOCK) = 8
43139 brk(NULL)                         = 0x5d72c7169000
43139 brk(0x5d72c718a000)               = 0x5d72c718a000
43139 openat(AT_FDCWD, "input.txt", O_RDONLY) = 3
43139 fstat(3, {st_mode=S_IFREG|0644, st_size=24, ...}) = 0
43139 read(3, "3 3\n1 1 1\n1 100 1\n1 1 1\n", 4096) = 24
43139 close(3)                          = 0
43139 fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
43139 fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
43139 read(0, "3\n2\n", 4096)           = 4
43139 rt_sigaction(SIGRT_1, {sa_handler=0x7a7deb899610, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7a7deb845330},
NULL, 8) = 0
43139 rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
43139 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7a7deafff000
43139 mprotect(0x7a7deb000000, 8388608, PROT_READ|PROT_WRITE) = 0
43139 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deb7ff990,
parent_tid=0x7a7deb7ff990, exit_signal=0, stack=0x7a7deafff000, stack_size=0x7fff80,
tls=0x7a7deb7ff6c0} => {parent_tid=[43140]}, 88) = 43140
43140 rseq(0x7a7deb7fffe0, 0x20, 0, 0x53053053 <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43140 <... rseq resumed>)               = 0
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43140 set_robust_list(0x7a7deb7ff9a0, 24 <unfinished ...>
43139 futex(0x7a7deb7ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43140, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43140 <... set_robust_list resumed>)    = 0
43140 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
43140 mmap(NULL, 134217728, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a7de2fff000
43140 munmap(0x7a7de2fff000, 16781312)  = 0
43140 munmap(0x7a7de8000000, 50327552)  = 0
43140 mprotect(0x7a7de4000000, 135168, PROT_READ|PROT_WRITE) = 0
43140 gettid()                          = 43140
43140 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
43140 fstat(3, {st_mode=S_IFREG|0644, st_size=42667, ...}) = 0
43140 mmap(NULL, 42667, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7a7debb64000
43140 close(3)                          = 0
43140 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
43140 read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832)
= 832
43140 fstat(3, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0
43140 mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a7debb33000
```

```
43140 mmap(0x7a7debb37000, 147456, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x7a7debb37000
43140 mmap(0x7a7debb5b000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x28000) = 0x7a7debb5b000
43140 mmap(0x7a7debb5f000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2b000) = 0x7a7debb5f000
43140 close(3)                          = 0
43140 mprotect(0x7a7debb5f000, 4096, PROT_READ) = 0
43140 munmap(0x7a7debb64000, 42667)     = 0
43140 futex(0x7a7debb60230, FUTEX_WAKE_PRIVATE, 2147483647) = 0
43140 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
43140 madvise(0x7a7deafff000, 8368128, MADV_DONTNEED) = 0
43140 exit(0)                           = ?
43139 <... futex resumed>)              = 0
43140 +++ exited with 0 +++
43139 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deb7ff990,
parent_tid=0x7a7deb7ff990, exit_signal=0, stack=0x7a7deafff000, stack_size=0x7fff80,
tls=0x7a7deb7ff6c0} => {parent_tid=[43141]}, 88) = 43141
43141 rseq(0x7a7deb7fffe0, 0x20, 0, 0x53053053 <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43141 <... rseq resumed>)               = 0
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43141 set_robust_list(0x7a7deb7ff9a0, 24 <unfinished ...>
43139 futex(0x7a7deb7ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43141, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43141 <... set_robust_list resumed>)    = 0
43141 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
43141 gettid()                          = 43141
43141 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
43141 madvise(0x7a7deafff000, 8368128, MADV_DONTNEED) = 0
43141 exit(0)                           = ?
43139 <... futex resumed>)              = 0
43141 +++ exited with 0 +++
43139 openat(AT_FDCWD, "output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
43139 fstat(3, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
43139 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deb7ff990,
parent_tid=0x7a7deb7ff990, exit_signal=0, stack=0x7a7deafff000, stack_size=0x7fff80,
tls=0x7a7deb7ff6c0} => {parent_tid=[43142]}, 88) = 43142
43142 rseq(0x7a7deb7fffe0, 0x20, 0, 0x53053053 <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43142 <... rseq resumed>)               = 0
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43142 set_robust_list(0x7a7deb7ff9a0, 24 <unfinished ...>
43139 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>
43142 <... set_robust_list resumed>)    = 0
43139 <... mmap resumed>)               = 0x7a7dea7fe000
43142 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43139 mprotect(0x7a7dea7ff000, 8388608, PROT_READ|PROT_WRITE) = 0
43142 <... rt_sigprocmask resumed>NULL, 8) = 0
43139 rt_sigprocmask(SIG_BLOCK, ~[],  <unfinished ...>
43142 gettid( <unfinished ...>
43139 <... rt_sigprocmask resumed>[], 8) = 0
43142 <... gettid resumed>)             = 43142
```

```
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deaffe990,
parent_tid=0x7a7deaffe990, exit_signal=0, stack=0x7a7dea7fe000, stack_size=0x7fff80,
tls=0x7a7deaffe6c0} <unfinished ...>
43142 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
43139 <... clone3 resumed> => {parent_tid=[43143]}, 88) = 43143
43143 rseq(0x7a7deaffefe0, 0x20, 0, 0x53053053 <unfinished ...>
43142 madvise(0x7a7deafff000, 8368128, MADV_DONTNEED <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43143 <... rseq resumed>)               = 0
43142 <... madvise resumed>)            = 0
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43139 futex(0x7a7deb7ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43142, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43143 set_robust_list(0x7a7deaffe9a0, 24 <unfinished ...>
43142 exit(0 <unfinished ...>
43143 <... set_robust_list resumed>)    = 0
43142 <... exit resumed>)               = ?
43143 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43139 <... futex resumed>)              = 0
43143 <... rt_sigprocmask resumed>NULL, 8) = 0
43142 +++ exited with 0 +++
43139 futex(0x7a7deaffe990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43143, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43143 gettid()                          = 43143
43143 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
43143 madvise(0x7a7dea7fe000, 8368128, MADV_DONTNEED) = 0
43143 exit(0)                           = ?
43139 <... futex resumed>)              = 0
43143 +++ exited with 0 +++
43139 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deaffe990,
parent_tid=0x7a7deaffe990, exit_signal=0, stack=0x7a7dea7fe000, stack_size=0x7fff80,
tls=0x7a7deaffe6c0} => {parent_tid=[43144]}, 88) = 43144
43144 rseq(0x7a7deaffefe0, 0x20, 0, 0x53053053 <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43144 <... rseq resumed>)               = 0
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43144 set_robust_list(0x7a7deaffe9a0, 24 <unfinished ...>
43139 rt_sigprocmask(SIG_BLOCK, ~[],  <unfinished ...>
43144 <... set_robust_list resumed>)    = 0
43139 <... rt_sigprocmask resumed>[], 8) = 0
43144 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43139
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLO
NE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7a7deb7ff990,
parent_tid=0x7a7deb7ff990, exit_signal=0, stack=0x7a7deafff000, stack_size=0x7fff80,
tls=0x7a7deb7ff6c0} <unfinished ...>
43144 <... rt_sigprocmask resumed>NULL, 8) = 0
43139 <... clone3 resumed> => {parent_tid=[43145]}, 88) = 43145
43144 gettid( <unfinished ...>
43145 rseq(0x7a7deb7fffe0, 0x20, 0, 0x53053053 <unfinished ...>
43139 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43144 <... gettid resumed>)             = 43144
43139 <... rt_sigprocmask resumed>NULL, 8) = 0
43145 <... rseq resumed>)               = 0
```

```
43139 futex(0x7a7deaffe990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43144, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43144 rt_sigprocmask(SIG_BLOCK, ~[RT_1],  <unfinished ...>
43145 set_robust_list(0x7a7deb7ff9a0, 24 <unfinished ...>
43144 <... rt_sigprocmask resumed>NULL, 8) = 0
43145 <... set_robust_list resumed>)    = 0
43145 rt_sigprocmask(SIG_SETMASK, [],  <unfinished ...>
43144 madvise(0x7a7dea7fe000, 8368128, MADV_DONTNEED <unfinished ...>
43145 <... rt_sigprocmask resumed>NULL, 8) = 0
43144 <... madvise resumed>)            = 0
43145 gettid( <unfinished ...>
43144 exit(0 <unfinished ...>
43145 <... gettid resumed>)             = 43145
43144 <... exit resumed>)               = ?
43139 <... futex resumed>)              = 0
43145 rt_sigprocmask(SIG_BLOCK, ~[RT_1],  <unfinished ...>
43144 +++ exited with 0 +++
43139 futex(0x7a7deb7ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 43145, NULL,
FUTEX_BITSET_MATCH_ANY <unfinished ...>
43145 <... rt_sigprocmask resumed>NULL, 8) = 0
43145 madvise(0x7a7deafff000, 8368128, MADV_DONTNEED) = 0
43145 exit(0)                           = ?
43139 <... futex resumed>)              = 0
43145 +++ exited with 0 +++
43139 getpid()                          = 43139
43139 getpid()                          = 43139
43139 write(3, "\nMatrix 3 x 3, window = 3, K = 2"..., 175) = 175
43139 close(3)                          = 0
43139 write(1, "Enter window size:\nEnter number "..., 116) = 116
43139 lseek(0, -1, SEEK_CUR)            = -1 ESPIPE (Illegal seek)
43139 exit_group(0)                     = ?
43139 +++ exited with 0 +++
```